Object Oriented Programming (OOP) using Python

In [2]:
```
class Rectangle:
    count = 0 #Class attribute

    def __init__(self, l, b): #Constructor
        self.length = l #Instance attribute
        self.breadth = b #Instance attribute
        Rectangle.count += 1

    def area(self): #Instance method
        return self.length * self.breadth

    def perimeter(self): #Instance method
        return 2 * (self.length + self.breadth)

    @classmethod
    def get_count(cls): #Class method
        return cls.count
```

In [4]:
```
Rectangle.get_count()
```

Out[4]: 0

In [6]:
```
Rectangle.count
```

Out[6]: 0

In [8]:
```
r1 = Rectangle(5,12)
```

In [10]:
```
r1.area()
```

Out[10]: 60

In [12]:
```
r1.perimeter()
```

Out[12]: 34

In [14]:
```
r1.get_count()
```

Out[14]: 1

In [16]:
```
r1.breadth
```

Out[16]: 12

In [18]:
```
r1.length
```

Out[18]: 5

In [20]:
```
r1.count
```

Out[20]: 1

In [22]:
```
r2 = Rectangle(10,10)
```

In [24]:
```
r2.length
```

Out[24]: 10

In [26]:
```
r2.breadth
```

Out[26]: 10

In [28]:
```
class Rectangle:
    count = 0 #Class attribute

    def __init__(self, l = 1, b = 1): #Constructor with default parameter
        self.length = l #Instance attribute
        self.breadth = b #Instance attribute
        Rectangle.count += 1

    def area(self): #Instance method
        return self.length * self.breadth

    def perimeter(self): #Instance method
        return 2 * (self.length + self.breadth)

    @classmethod
    def get_count(cls): #Class method
        return cls.count
```

In [30]:
```
r3 = Rectangle()
```

In [32]:
```
r3.length
```

Out[32]: 1

In [34]:
```
r3.breadth
```

Out[34]: 1

In [36]:
```
r4 = Rectangle(5)
```

In [38]:
```
r4.length
```

Out[38]: 5

In [40]:
```
r4.breadth
```

Out[40]: 1

In [42]:
```
r4.area()   #-> Rectangle.area(r1)
```

Out[42]: 5

In [44]:
```
Rectangle.area(r4)
```

Out[44]: 5

In [46]:
```
class Student:
    No_of_students = 0

    def __init__(self, name, rollno, div):
        self.Name = name
        self.Roll_No = rollno
        self.Division = div
        Student.No_of_students += 1

    def show(self):
        print("ID of self : ",id(self))
        print("Name :",self.Name)
        print("Roll No :",self.Roll_No)
        print("Division : ",self.Division)

    def __str__(self):  #returns a string when object is printed
        return str(self.Roll_No)
```

In [48]:
```
Student.No_of_students
```

Out[48]: 0

In [50]:
```
s1 = Student("Darshan", 301, "B")
```

In [52]:
```
s1.No_of_students
```

Out[52]: 1

In [54]:
```
Student.No_of_students = 200
```

In [56]:
```
s1.No_of_students
```

Out[56]: 200

In [58]:
```
s1.No_of_students = 520
```

In [60]:
```
Student.No_of_students
```

Out[60]: 200

In [62]:
```
id(s1)
```

Out[62]: 2735416331056

In [64]:
```
s1.show()
```

```
ID of self :  2735416331056
Name : Darshan
Roll No : 301
Division :  B
```

In [66]:
```
print(s1)
```

```
301
```

In [68]:
```
class MathUtility:
    @staticmethod
    def add(a,b):
        return a + b
```

In [70]:
```
MathUtility.add(10,20)
```

Out[70]: 30

In [72]:
```
m1 = MathUtility()
```

In [74]:
```
m1.add(4,5)
```

Out[74]: 9

Encapsulation (Access Modifiers)

public : accessible everywhere (inside and outside the class & in subclasses)

by default every attribute & method is public

In [78]:
```
class Employee:
    company_name = "TCS"
    def __init__(self, name, city, income):
        self.EName = name
        self.City = city
        self.Annual_Income = income

    def TaxCalculation(self):
        return self.Annual_Income * 0.2

    def ShowTaxDetails(self):
        print("Tax to be paid :",self.TaxCalculation())
```

In [80]:
```
emp1 = Employee("Darshan","Rajkot",500000)
```

In [82]:
```
emp1.Annual_Income
```

Out[82]: 500000

In [84]:
```
emp1.ShowTaxDetails()
```

```
Tax to be paid : 100000.0
```

In [86]:
```
class TA(Employee):
    def show(self):
        print(super().company_name)
```

In [88]:
```
ta1 = TA("Jigar","Morbi",10000)
```

In [90]:
```
ta1.show()
```

```
TCS
```

private : accessible only inside the class, not outside the class or not even in subclasses

attribute or method can be made private by specifying __ (double underscore) before their names

but still can be accessed using _ClassName.__Attribute/MethodName (Name Mangling)

In [93]:
```
class Employee:
    __company_name = "TCS"
    def __init__(self, name, city, income):
        self.EName = name
        self.City = city
        self.Annual_Income = income

    def __TaxCalculation(self):
        return self.Annual_Income * 0.2

    def ShowTaxDetails(self):
        print("Tax to be paid :",self.__TaxCalculation())
```

In [95]:
```
Employee.__company_name
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[95], line 1
----> 1 Employee.__company_name

AttributeError: type object 'Employee' has no attribute '__company_name'
```

In [104…
```
e1 = Employee("Ajay", "Rajkot", 100000)
```

In [106…
```
e1.__TaxCalculation()   #Employee.TaxCalculation(e1)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[106], line 1
----> 1 e1.__TaxCalculation()

AttributeError: 'Employee' object has no attribute '__TaxCalculation'
```

In [108…
```
e1.ShowTaxDetails()
```

```
Tax to be paid : 20000.0
```

In [110…
```
e1._Employee__company_name  # Name Mangling
```

Out[110…
```
'TCS'
```

In [112…
```
e1._Employee__TaxCalculation()
```

Out[112…
```
20000.0
```

In [114…
```
class TA(Employee):
    def show(self):
```

```
        print(super().__company_name)
```

In [116…
```
ta1 = TA("Jigar","Morbi",10000)
```

In [118…
```
ta1.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[118], line 1
----> 1 ta1.show()

Cell In[114], line 3, in TA.show(self)
      2 def show(self):
----> 3     print(super().__company_name)

AttributeError: 'super' object has no attribute '_TA__company_name'
```

protected : accessible inside the class and it's subclasses, not outside the class

attribute or method can be made private by specifying _ (single underscore) before their names

still they can be accessed outside and inside the subclasses, as Python doesn't have strict enforcement (not recommended to access)

In [121…
```
class Employee:
    _company_name = "TCS"
    def __init__(self, name, city, income):
        self.EName = name
        self.City = city
        self.Annual_Income = income

    def _TaxCalculation(self):
        return self.Annual_Income * 0.2

    def ShowTaxDetails(self):
        print("Tax to be paid :",self._TaxCalculation())
```

In [123…
```
e1 = Employee("Ajay", "Rajkot", 100000)
```

In [125…
```
e1._company_name
```

Out[125…
```
'TCS'
```

In [127…
```
e1._TaxCalculation()
```

Out[127…
```
20000.0
```

Inheritance & it's types

In [130…
```
#syntax
class Parent:
```

```
    pass

class Child(Parent):
    pass
```

Single Inhertance

In [133…

```
class A:
    def displayA(self):
        print("Class-A")
class B(A):
    def displayB(self):
        print("Class-B")

obj = B()
obj.displayA()
obj.displayB()
```

```
Class-A
Class-B
```

Multiple Inheritance

In [136…

```
class A:
    def displayA(self):
        print("Class-A")

class B:
    def displayB(self):
        print("Class-B")

class C(A,B):
    def displayC(self):
        print("Class-C")

obj = C()
obj.displayA()
obj.displayB()
obj.displayC()
```

```
Class-A
Class-B
Class-C
```

Multilevel Inheritance

In [139…

```
class A:
    def displayA(self):
        print("Class-A")

class B(A):
    def displayB(self):
        print("Class-B")

class C(B):
    def displayC(self):
        print("Class-C")
```

```
obj = C()
obj.displayA()
obj.displayB()
obj.displayC()
obj1 = B()
obj1.displayA()
obj1.displayB()
obj2 = A()
obj2.displayA()
```

```
Class-A
Class-B
Class-C
Class-A
Class-B
Class-A
```

Hierarchical Inheritance

In [142…

```
class A:
    def displayA(self):
        print("Class-A")

class B(A):
    def displayB(self):
        print("Class-B")

class C(A):
    def displayC(self):
        print("Class-C")

obj = C()
obj.displayA()
obj.displayC()
obj.displayB()
```

```
Class-A
Class-C
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[142], line 16
     14 obj.displayA()
     15 obj.displayC()
---> 16 obj.displayB()

AttributeError: 'C' object has no attribute 'displayB'
```

Hybrid Inheritance

In [145…

```
class A:
    def displayA(self):
        print("Class-A")

class B(A):
    def displayB(self):
        print("Class-B")

class C(A):
    def displayC(self):
```

```
        print("Class-C")

class D(B,C):
    def displayD(self):
        print("Class - D")

obj = D()
obj.displayA()
obj.displayC()
obj.displayB()
obj.displayD()
```

```
Class-A
Class-C
Class-B
Class - D
```

Method Overriding

In [148…

```
class A:
    def display(self):
        print("Class-A")

class B(A):
    def display(self):
        print("Class-B")
```

In [150…

```
obj = B()
obj.display()
```

```
Class-B
```

In [152…

```
class A:
    def display(self):
        print("Class-A")

class B(A):
    def display(self):
        super().display()
        #A.display(self)
        print("Class-B")
```

In [154…

```
obj = B()
obj.display()
```

```
Class-A
Class-B
```

In [156…

```
class A:
    def display(self):
        print("Class-A")

class B(A):
    def display(self):
        super().display()
        #A.display(self)
        print("Class-B")

class C(B):
```

```
    def display(self):
        super().display()
        #B.display(self)
        print("Class-C")
```

In [158…

```
obj = C()
obj.display()
```

```
Class-A
Class-B
Class-C
```

In [160…

```
C.mro()
```

Out[160…

```
[__main__.C, __main__.B, __main__.A, object]
```

In [162…

```
class Rectangle:
    def __init__(self,l,b):
        self.Length = l
        self.Breadth = b
```

In [164…

```
class Cuboid(Rectangle):
    def __init__(self,l,b,h):
        super().__init__(l,b)
        self.Height = h

    def show(self):
        print("Length :",self.Length)
        print("Breadth :",self.Breadth)
        print("Height :",self.Height)
```

In [166…

```
cub1 = Cuboid(5,10,2)
```

In [168…

```
cub1.show()
```

```
Length : 5
Breadth : 10
Height : 2
```

Polymorphism

Operator Overloading

In [172…

```
10 + 20
```

Out[172…

```
30
```

In [174…

```
"10" + "20"
```

Out[174…

```
'1020'
```

In [176…

```
[1,2,3] + [4,5,6]
```

Out[176…

```
[1, 2, 3, 4, 5, 6]
```

In [178…

```
(1,2) + (3,)
```

Out[178… (1, 2, 3)

In [180…
```
5 * 2
```

Out[180… 10

In [182…
```
"Hello" * 3
```

Out[182… 'HelloHelloHello'

Method Overloading

In [187…
```
class Maths:
    def add(self,a):
        return a

    def add(self,a,b):
        return a + b

    def add(self,a,b,c):
        return a + b + c
```

In [189…
```
m1 = Maths()
```

In [191…
```
m1.add(10)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[191], line 1
----> 1 m1.add(10)

TypeError: Maths.add() missing 2 required positional arguments: 'b' and 'c'
```

In [193…
```
m1.add(10,20)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[193], line 1
----> 1 m1.add(10,20)

TypeError: Maths.add() missing 1 required positional argument: 'c'
```

In [195…
```
m1.add(10,20,30)
```

Out[195… 60

In [197…
```
class Maths:
    def add(self,a=0,b=0,c=0): #using default arguments
        return a + b + c
```

In [199…
```
m1 = Maths()
```

In [201…
```
m1.add()
```

Out[201… 0

In [203…
```
m1.add(10)
```
Out[203…
```
10
```
In [205…
```
m1.add(10,20)
```
Out[205…
```
30
```
In [207…
```
m1.add(10,20,30)
```
Out[207…
```
60
```
In [209…
```
m1.add(10,20,30,40)
```
```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[209], line 1
----> 1 m1.add(10,20,30,40)

TypeError: Maths.add() takes from 1 to 4 positional arguments but 5 were given
```
In [211…
```
class Maths:
    def add(self,*args): #using variable length arguments
        return sum(args)
```
In [213…
```
m1 = Maths()
```
In [215…
```
m1.add(1,2,3,4,5,6)
```
Out[215…
```
21
```
In [217…
```
m1.add()
```
Out[217…
```
0
```

Abstraction

In [220…
```
# ABC - class
# @abstractmethod - method decorator

# abc (abstract base class) - module
```
In [222…
```
from abc import ABC, abstractmethod
```
In [224…
```
class Shape(ABC): #now Shape is an abstract class

    @abstractmethod
    def area(self): #abstract method
        pass

    def greet(self): #concrete method
        print("Hello.")
```
In [226…
```
s1 = Shape()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[226], line 1
----> 1 s1 = Shape()

TypeError: Can't instantiate abstract class Shape without an implementation for a
bstract method 'area'
```

In [228…

```
class Rectangle(Shape):
    def __init__(self,l,b):
        self.length = l
        self.breadth = b

    # def area(self):
    #     return self.length * self.breadth

    def show(self):
        print("Length :",self.length)
        print("Breadth :",self.breadth)
```

In [230…

```
r1 = Rectangle(1,2)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[230], line 1
----> 1 r1 = Rectangle(1,2)

TypeError: Can't instantiate abstract class Rectangle without an implementation f
or abstract method 'area'
```

In [232…

```
class Rectangle(Shape):
    def __init__(self,l,b):
        self.length = l
        self.breadth = b

    def area(self):
        return self.length * self.breadth

    def show(self):
        print("Length :",self.length)
        print("Breadth :",self.breadth)
```

In [234…

```
r1 = Rectangle(1,2)
```

In [236…

```
r1.show()
```

```
Length : 1
Breadth : 2
```

In [238…

```
r1.area()
```

Out[238…

```
2
```

In [240…

```
class Shape(ABC): #now Shape is an interface as all the methods are abstract met

    @abstractmethod
    def area(self): #abstract method
        pass
```

```
    @abstractmethod
    def perimeter(self): #abstract method
        pass
```

In [242…

```
s1 = Shape()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[242], line 1
----> 1 s1 = Shape()

TypeError: Can't instantiate abstract class Shape without an implementation for a
bstract methods 'area', 'perimeter'
```

In [244…

```
class Rectangle(Shape):
    def __init__(self,l,b):
        self.length = l
        self.breadth = b

    def area(self):
        return self.length * self.breadth

    def perimeter(self):
        return 2 * (self.length + self.breadth)
```

In [246…

```
r1 = Rectangle(5,3)
```

In [248…

```
r1.area()
```

Out[248…

```
15
```

In [250…

```
r1.perimeter()
```

Out[250…

```
16
```

In [252…

```
import math
```

In [254…

```
class Circle(Shape):
    def __init__(self,r):
        self.radius = r

    def area(self):
        return math.pi * self.radius * self.radius

    # def perimeter(self):
    #     return 2 * math.pi * self.radius
```

In [256…

```
c1 = Circle(5)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[256], line 1
----> 1 c1 = Circle(5)

TypeError: Can't instantiate abstract class Circle without an implementation for
abstract method 'perimeter'
```

In [258…

```
class Circle(Shape):
    def __init__(self,r):
        self.radius = r

    def area(self):
        return math.pi * self.radius * self.radius

    def perimeter(self):
        return 2 * math.pi * self.radius
```

In [260…

```
c1 = Circle(5)
```

In [262…

```
c1.area()
```

Out[262…

```
78.53981633974483
```

In [264…

```
c1.perimeter()
```

Out[264…

```
31.41592653589793
```

In [266…

```
c1.radius
```

Out[266…

```
5
```